Aran Learn 3: Cryptography Basics

For: Inbathamizhan and Aasif in full; **everyone reads section 4** (the four ways to protect a value). **Time:** about 7 hours with practice. **Before this:** Learn 1 and Learn 2.

Cryptography is how Aran makes a stolen log useless to the thief, gives each destination its own tokens, and erases a customer everywhere at once. This doc builds it up from zero, with runnable Python for every idea.

1. What cryptography is for

Security people describe what they protect with three words. Each Aran mechanism serves at least one.

Goal	Plain meaning	Tool that provides it	Where Aran uses it
Confidentiality	Only the right people can read the data	Encryption	FF1 tokens, vault storage, per-customer keys
Integrity	Nobody can change the data without being noticed	Hashes, HMAC, authenticated encryption	Tamper-evident erasure log
Authenticity	You can prove who produced the data	HMAC, digital signatures	Signed erasure receipts

Kerckhoffs's principle (1883). A system must stay secure even if the attacker knows exactly how it works; only the key is secret. That is why Aran publishes its algorithms (Verhoeff, FF1, AES) and spends its effort on protecting keys. If the security of a design depends on nobody reading the code, it is not secure.

2. Core building blocks

2.1 Keys and randomness

A key is a secret random number, usually 128 or 256 bits. "256 bits" means 2^256 possible keys, more than the number of atoms in the observable universe, so guessing is hopeless. The key must be truly unpredictable, which means using the operating system's secure random source.

```
import secrets, random
key = secrets.token_bytes(32)      # 256-bit key: correct
otp = secrets.randbelow(10**6)     # a 6-digit OTP: correct
bad = random.randint(0, 10**6)     # WRONG for secrets: 'random' is predictable
```

Python's `random` module is designed for simulations. Given a few hundred outputs, its internal state can be reconstructed and every future value predicted. Use `secrets` for anything an attacker could benefit from guessing.

2.2 Symmetric and asymmetric cryptography

Kind	Keys	Speed	Example	Aran use
Symmetric	One shared secret key locks and unlocks	Very fast	AES	Tokens, storage encryption
Asymmetric (public key)	A public key anyone may have, and a private key kept secret	Slower	RSA, Ed25519	Signing erasure receipts so a customer can verify them

Think of symmetric as a house key that both you and your family hold, and asymmetric as a letterbox: anyone can drop a letter in (public key), only the owner can open it (private key). Signatures work the other way round: only the owner can sign, anyone can check.

2.3 AES, the standard block cipher

AES (Advanced Encryption Standard, chosen by NIST in 2001) takes a 16-byte block and a key, and outputs 16 scrambled bytes. With the same key it can unscramble them. On its own it only handles exactly 16 bytes, so it is always used inside a **mode** that handles longer data.

Mode	What it does	Verdict
ECB	Encrypts each 16-byte block separately	Never use for data: identical blocks give identical ciphertext, so patterns show through. FF1 uses AES-ECB only internally, on one block at a time, which is safe
CBC	Chains blocks together	Old; needs a separate integrity check
GCM	Encrypts and adds a 16-byte authentication tag	The default choice: it detects any tampering

2.4 Authenticated encryption with AES-GCM

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

key = AESGCM.generate_key(bit_length=256)
aes = AESGCM(key)
nonce = os.urandom(12)                    # must never repeat with the same key
ct = aes.encrypt(nonce, b"234123412346", b"cust-101|marketing")
len(ct)                                   # 28 = 12 bytes of data + 16-byte tag

aes.decrypt(nonce, ct, b"cust-101|marketing")   # b'234123412346'
```

The third argument is **associated data**: it is not encrypted but is bound to the ciphertext. If anyone changes even one bit of the ciphertext, or tries to decrypt it with a different associated label, decryption fails with `InvalidTag` instead of returning wrong data. Aran puts the customer and purpose in the associated data, so a marketing ciphertext can never be passed off as a fraud-team record.

The nonce rule. A nonce is a "number used once". Reusing a nonce with the same key in GCM can reveal the key's authentication secret and let an attacker forge messages. Use a fresh random 12-byte nonce every time and store it next to the ciphertext; it does not need to be secret.

2.5 Hashing

A hash function such as SHA-256 turns any input into a fixed 32-byte fingerprint. It has three properties:

1. **One way:** from the hash you cannot work back to the input.
2. **Collision resistant:** nobody can find two inputs with the same hash.
3. **Avalanche:** changing one character changes the output completely.

```
import hashlib
hashlib.sha256(b"234123412346").hexdigest()[:16]   # '2e3f18a222de50f7'
hashlib.sha256(b"234123412347").hexdigest()[:16]   # 'c5bf013831d38051'
```

One digit changed, and the two fingerprints share nothing.

2.6 Why a plain hash fails for Aadhaar, and what HMAC fixes

"One way" only helps when the input space is huge. Aadhaar numbers are 12 digits with the first digit 2 to 9 and one check digit, so there are only 8 × 10^10 of them. An attacker who steals a table of plain SHA-256 hashes can simply hash every possible Aadhaar and look up the matches. On one laptop core plain Python manages about 1.7 million hashes per second, so the whole space takes about 13 hours; optimised code on a gaming GPU takes minutes.

HMAC mixes a secret key into the hash. Without the key, the attacker cannot compute the right fingerprints, so the dictionary attack is impossible.

```
import hmac, hashlib
hmac.new(b"secret-kept-in-HSM", b"234123412346", hashlib.sha256).hexdigest()[:16]
# '8d7357a81529fbe3'
hmac.new(b"another-key", b"234123412346", hashlib.sha256).hexdigest()[:16]
# '455c09e7f7fd1942'   same input, different key, unrelated output
```

Aran's own-customer registry (Learn 4) stores HMAC values, never plain hashes, and the HMAC key never leaves the HSM. When comparing two HMAC values in code, use `hmac.compare_digest(a, b)` rather than `==`, so an attacker cannot learn how many leading characters matched from how long the comparison took.

3. Keys in a bank: HSMs and envelope encryption

HSM (hardware security module). A tamper-resistant device that stores keys and performs cryptography inside itself. Applications send it data and get results back; the key itself never leaves. If someone opens the case, it wipes its keys. Banks already use HSMs for card PINs, and UIDAI requires one for the Aadhaar Data Vault.

Envelope encryption. An HSM is fast but not fast enough for every record, and it has limited storage. So banks use two layers:

Layer	Name	Lives where	Encrypts what
Top	Key-encryption key (KEK)	Inside the HSM, never exported	The data keys
Bottom	Data-encryption key (DEK)	Stored next to the data, but only in encrypted (wrapped) form	The actual records

To read a record, the application asks the HSM to unwrap the DEK, uses it, and then forgets it. Aran's per-customer, per-purpose keys are DEKs wrapped by an HSM KEK. Destroying a DEK is cheap and instant, which is what makes crypto-shredding (section 6) practical at the scale of crores of customers.

4. Four ways to protect a value (everyone reads this)

Aran chooses one protection for each field and each destination. They differ in two questions: can the real value come back, and is the shape kept?

Method	Output for 2341 2341 2346	Can the real value come back?	Keeps the 12-digit shape?	Use it when
Masking	XXXX XXXX 2346	No	Yes	A person only needs to recognise the number
Vault tokenisation	a random 12-digit token, for example 9017 4423 8150	Yes, by asking the vault	Yes	Systems must join records but should not see the value
Format-preserving encryption (FF1)	4187 2509 6699 (with an all-zero demo key and tweak "crm")	Yes, with the key	Yes	No vault is possible and the database schema cannot change
Normal encryption (AES-GCM)	28 bytes that look random	Yes, with the key	No	Storage where the shape does not matter

4.1 Masking

Masking replaces part of the value with a fixed character. It is simple and cannot be reversed, but it is not automatically safe: Learn 4 shows that a card masked as first 8 and last 4 leaves only 1,000 possibilities.

4.2 Vault tokenisation, with an example

The vault is a guarded table. The token is random, so it carries no information about the real value; only the vault can map it back.

Token (stored everywhere)	Encrypted real value (only in the vault)	Customer key id
9017 4423 8150	AES-GCM ciphertext of 2341 2341 2346	cust-101/kyc
3360 1185 2097	AES-GCM ciphertext of another Aadhaar	cust-205/kyc

The CRM, the logs and the analytics system store only the left column. A breach of any of them leaks tokens, not Aadhaar numbers. This is what UIDAI's Aadhaar Data Vault rule asks banks to

do with "reference keys". Aran's vault tokens also carry a deliberately wrong check digit (Learn 2, section 6), so they can never be confused with a real Aadhaar.

4.3 Format-preserving encryption (FF1)

FF1 is a NIST standard (SP 800-38G) that encrypts a string of digits into a string of digits of the same length. Unlike a vault, it needs no table: anyone with the key can reverse it.

How it works inside: a Feistel network. Split the number into a left half L and a right half R. In each round, compute a scrambled value from R and the key, add it to L, and swap the halves. Undoing a round only needs the same scrambled value, subtracted instead of added, so the whole thing can be reversed even though the scrambling function itself cannot.

A toy version with a made-up round function F, on the 6-digit number 482913 with key 42, makes this concrete:

```
        def F(x, k, r):                       # toy round function; FF1 uses AES here
            return (x * 31 + k * 7 + r * 13) % 1000

        def enc(n, k):
            L, R = divmod(n, 1000)            # 482913 -> L = 482, R = 913
            for r in range(4):
                L, R = R, (L + F(R, k, r)) % 1000
            return L * 1000 + R

        def dec(n, k):
            L, R = divmod(n, 1000)
            for r in reversed(range(4)):
                L, R = (R - F(L, k, r)) % 1000, L
            return L * 1000 + R

        enc(482913, 42)     # 138280
        dec(138280, 42)     # 482913
```

Round	L	R	F(R)	New L	New R = (L + F) mod 1000
0	482	913	597	913	79
1	913	79	756	79	669
2	79	669	59	669	138
3	669	138	611	138	280

Output: 138 280. Never use this toy for real data: F here is simple arithmetic that anyone can invert. Real FF1 does 10 rounds and builds F from AES, which is why the result is secure.

The tweak. FF1 takes an extra public input called a tweak. Changing the tweak gives a completely different output for the same input and key. Aran uses the destination name as the tweak, which is how each destination gets its own tokens:

```
from aran_core import FF1
ff1 = FF1(key=bytes(16))                # demo key only
ff1.encrypt("23412341234", tweak=b"crm")        # '55258817336'
ff1.encrypt("23412341234", tweak=b"logs")       # '78936098212'
ff1.decrypt("55258817336", tweak=b"crm")        # '23412341234'
```

Aran's FF1 in `research/aran_core.py` matches NIST's published sample answers exactly; `test_core.py` checks this.

One rule to remember. Never use FF1 on tiny fields such as a 4-digit last-4 or a 6-digit OTP. With only 10,000 or 1,000,000 possible values, an attacker who can encrypt or decrypt a few values can map the whole domain. NIST's revision drafts of SP 800-38G raise the minimum domain size (to one million values in the 2019 draft). For OTPs, Aran does not tokenise at all: it detects and drops them.

4.4 Choosing between them

Question	If yes
Does anyone downstream need the real value back?	No: mask. Yes: continue
Can every reverse lookup go through a central service?	Yes: vault tokenisation. No: continue
Must the value keep its exact length and digit-only format?	Yes: FF1 (field must be large). No: AES-GCM

5. Merkle trees: tamper-evident logs

A Merkle tree hashes records in pairs, then hashes the pairs, and so on until one **root** hash remains. Changing any record changes the root.

```
          root
        /      \
    H(a|b)    H(c|d)
    /   \      /   \
H(r1) H(r2) H(r3) H(r4)
```

```
import hashlib
def H(b): return hashlib.sha256(b).digest()

def merkle_root(items):
    level = [H(x.encode()) for x in items]
    while len(level) > 1:
        if len(level) % 2:                  # odd count: repeat the last one
            level.append(level[-1])
        level = [H(level[i] + level[i + 1]) for i in range(0, len(level), 2)]
    return level[0].hex()

recs = ["erase cust-101 marketing", "erase cust-205 marketing",
        "erase cust-101 servicing", "erase cust-333 fraud"]
merkle_root(recs)[:16]      # '2697211a205a421d'
recs[1] = "erase cust-206 marketing"
merkle_root(recs)[:16]      # '3545de5a0c3824f8'  one change, new root
```

Inclusion proofs. To prove that record r2 is in the tree, you do not need to show all the other records. You give H(r1) and H(c|d). The verifier computes H(r2), combines it with H(r1) to get H(a|b), combines that with H(c|d) and checks it equals the published root. For a log of a million records the proof is only 20 hashes, because log2(1,000,000) ≈ 20. Certificate Transparency, which keeps web certificates honest, uses the same structure (RFC 6962).

6. Erasure by destroying keys

The DPDP Act gives customers the right to erasure (Learn 7). Deleting database rows does not reach log files, backups, data-lake copies or vendor systems. Aran erases by destroying keys instead.

Crypto-shredding. Encrypt each customer's data with that customer's own key. To erase the customer, destroy the key. Every copy of their encrypted data, wherever it went, becomes unreadable at once, including copies in backups you cannot edit.

Why Aran's version is stronger. Aran's vault tokens are random, and the only link from a token to the real value is the vault entry encrypted under the customer's key. After shredding, the token says nothing at all about the customer, because there is no maths path from a random number back to a person. This is Theorem 3 in the research doc, based on Shannon's perfect secrecy.

Purpose-bound keys, worked example. Laws sometimes force a bank to keep data, such as KYC records, for years. So Aran gives each customer one key per purpose:

Key	Protects	When it is destroyed
cust-101/marketing	Offers, campaign data	When the customer withdraws consent, immediately
cust-101/servicing	Account servicing records	When the account is closed and the purpose is served
cust-101/fraud	Fraud-analytics features	At the end of the fraud retention period
cust-101/kyc	KYC documents	Automatically when the legal KYC retention period ends

When customer 101 says "stop using my data for marketing", Aran destroys only the marketing key now, writes the destruction into the Merkle log, and gives the customer a receipt: the log entry, its inclusion proof and the bank's signature over the root. The customer, or the Data Protection Board, can verify the receipt without trusting the bank's word.

7. Common mistakes to avoid

Mistake	Why it is dangerous	Do this instead
Writing your own cipher	Home-made crypto is almost always breakable	Use AES, FF1, HMAC from the `cryptography` library
Keys in code or in git	Anyone with the repo has the key, forever	Keys in an HSM or a secrets manager; demo keys clearly marked
Plain SHA-256 of identifiers	Small input space, dictionary attack	HMAC with an HSM key
Reusing a GCM nonce	Breaks integrity and can leak data	Fresh random 12-byte nonce each time
`random` for OTPs or tokens	Predictable	`secrets`
FF1 on short fields	Whole domain can be mapped	Drop, mask, or use a vault
Comparing MACs with ==	Timing leak	`hmac.compare_digest`

8. Practice

- **P1.** Compute the SHA-256 and the HMAC of "hello". Change one letter and compare. Change the HMAC key and compare.
- **P2.** Time how many SHA-256 hashes per second your laptop does in Python. Estimate how long hashing every Aadhaar would take.
- **P3.** Encrypt "234123412346" with AES-GCM using associated data "cust-1|kyc", then try to decrypt with "cust-1|marketing". What happens and why is that good?
- **P4.** Run the toy Feistel code. Change it to 6 rounds and check that `dec(enc(x))` still returns x for 1,000 random x.
- **P5.** Encrypt an 11-digit string with `FF1` using two different tweaks, and confirm both decrypt back.
- **P6.** Extend `merkle_root` into a function that returns the inclusion proof for item i, and a second function that verifies it.
- **P7.** Explain to a teammate, in three sentences, why destroying cust-101/marketing makes that customer's marketing tokens meaningless in every log, backup and vendor copy.

9. Further reading

- Dan Boneh and Victor Shoup, "A Graduate Course in Applied Cryptography" (free online), chapters 1 to 4 and 8.
- Christof Paar and Jan Pelzl, "Understanding Cryptography", with free video lectures, for AES and modes.
- NIST SP 800-38G for FF1; NIST SP 800-38D for GCM.
- RFC 6962 sections 2.1 and 2.1.1 for Merkle inclusion proofs.
- The `cryptography` library documentation at cryptography.io.